cidrtools API Reference

Release 1.2.0

Gene C

Aug 22, 2026

C API REFERENCE:

CHAPTER

ONE

CIDRTOOLS

1.1 Overview

Cidrtools is a library with a collection of tools that examine and manipulate network cidr blocks. The functions cover many common tasks and are designed to be extremely fast. These can be called directly from C as well as other languages that can bind to compiled shared libraries.

The source is available in Github cidrtools as well as the Arch AUR.

There is a companion cidrtools-cffi package providing Python bindings to the library. This module is significantly faster than a pure Python implementation using the *ipaddr* module.

1.2 Recent Changes

1.2.0

- Add ct_cidrs_intersection(). Computes the intersecting subnet(s) of two sets of cidr blocks.

1.1.0

- Bug fix: Original code assumed excluded cidrs are subnets. Enhance code to handle excluded networks being supernets.

1.3 hostcheck application

The package includes the *hostcheck* application which uses the library.

It takes a cidr block as an argument and prints a sample of IP addresses that are taken from the range of IPs spanned by the cidr block.

It also displays the hostname of each IP if DNS provides one.

The application ues three cidrtool functions:

- ct_cidr_to_range() - get the first and last IP in the cidr block
- ct_ip_str_to_hostname() - DNS PTR lookup.
- ct_ip_address_increment() - Find the IP address a number of IPs past an IP.

Source is in the *apps* directory and a man page is also provided.

1.4 Compact Benchmark

This benchmark has each implementation *compact* the same random set of 100,000 cidr blocks. This is repeated with different random sets. Some sets compact a lot and some very little.

While there is variation across trial sets, results are pretty consistent. The actual time to compact the set of cidr blocks using CFFI bindings is close to the raw C-code. The remainder of the time in the Python module is the overhead crossing from Python to C and back. The overhead is in around 35% or so. In absolute time this is still pretty small.

For example to compact one set 100,000 cidr blocks that compacts down to 54,941 cidr blocks:

```
C-code/compact only     0.012630286 seconds
Cidrtools-CFFI          0.025331798 seconds
Python/ipaddress        0.937566086 seconds     37 x slower
```

The take away, from a performance perspective, is to use compiled code and minimize the role Python itself plays in some performance sensitive areas.

One alternative to CFFI is using Cython code directly. This produces it's own compiled C-code and also has a significant performance boost over pure Python, but we found this approach complicated, brittle and less maintainable than having core functions written in pure C.

Cython can also be used to bind to the library as can Py03.

1.5 Code Checks

The static code checker is run using:

```
./scripts/check-source
```

The unit test suite can be run with or without *valgrind*. In both caes all tests should complete without any errors.

```
./scripts/do-build debug
./scripts/run-tests valgrind
```

To run the tests standard tests without using valgrind, do:

```
./scripts/do-build
./scripts/run-tests
```

1.6 AI Tooling

Assisted by:

- Anthropic's Claude
- Google's Gemini

CHAPTER

TWO

CIDRTOOLS C API REFERENCE

2.1 Data Structures

struct `ct_address`

A single IP address. It can be either an `IPv4` or an `IPv6` address.

It has two elements:

- **family** : One of `AF_INET` for `IPv4` or `AF_INET6` for `IPv6`.
- **add** : This is a union of struct in_addr and struct in6_addr. The union combines an uint32_t with an array uint8_t[16] in linux.

struct `ct_cidr`

A single CIDR block.

A cidr block is an IP Address and a prefix.

- **addr** : The IP address
- **prefix** : The prefix which is capped at 32 or 128 for `IPv4` or `IPv6`

struct `ct_cidrs`

This container struct holds an array of cidr blocks and a count of the number of CtCidr elements in the array.

The blocks are allocateed as an array of structs. To allocate `count` cidr blocks :

ct_allocate_cidrs(count, cirs)

- **count** : The number of cidrs.
- **blocks** : Array ic cidr block structs.

2.2 Core Functions

Check If CIDR contained Contained within Other CIDR(s)

bool `ct_cidr_contains_cidr`(const CtCidr *parent, const CtCidr *target)

Checks whether one cidr block is contained within another.

Parameters

- `target` – The cidr to check
- `parent` – The parent cidr.

Returns

true if the target network is within the parent network.

bool `ct_cidr_is_subnet`(const CtCidr *cidr, const CtCidrs *cidrs)

Checks an whether a cidr block is contained in any network in a list of cidrs.

Parameters

- `cidr` – The input cidr to check.
- `cidrs` – The list of cidrs to check agains.

Returns

true if the target block is enclosed inside any block within the array list.

bool `ct_cidr_contains_ip`(const CtCidr *cidr, const CtAddress *ip)

Determines if an `IP` address falls inside the `IP` range spanned by a cidr block.

Convenience wrapper of ct_cidr_contains_cidr().

Parameters

- `ip` – The `IP` to check.
- `cidr` – The cidr block.

Returns

true if the IP lies inside the subnet boundaries, false otherwise.

Get Host Bits of an IP Address

static inline int `ct_get_host_bits_v4`(const CtCidr *cidr, CtAddress *addr)

Internal helper to isolate IPv4 host bits.

int `ct_get_host_bits`(const CtCidr *cidr, CtAddress *addr)

Extracts the host bits of a cidr block. The bits past the prefix boundary.

Parameters

- `cidr` – The cidr to examine.
- `addr` – The output which has the host bits.

Returns

0 on success, or -1 on invalid parameters/prefixes.

char *`ct_format_host_bits`(const CtCidr *cidr)

Extracts host bits of a cidr block and returns a formatted string.

Returns allocated string, caller responsible for free()'ing the mem.

Uses ct_get_host_bits() and ct_ip_address_to_str().

Parameters

- `cidr` – The cidr to examine.

Returns

String containing the formatted host bits.

Exract the IP Address and Prefix from a CIDR

int `ct_str_to_cidr_parts`(const char *str, char *ip_buf, size_t ip_buflen, uint8_t *prefix)

Splits an input string into an IP address component and a prefix component.

If there is no prefix in the input string, prefix will be set to 32 for `IPv4` or 128 for `IPv6` Caller should then set the prefix based on family.

Safely strips any trailing whitespace characters from both prefix and raw IP blocks.

Parameters

- `str` – The input CIDR string (e.g., "192.168.1.1/24 n" or "10.0.0.1 tn").
- `ip_buf` – Output buffer to save the isolated IP string block.
- `ip_buflen` – Size limit of the destination ip_buf.
- `prefix` – Output pointer to hold the parsed uint8_t prefix value.

Returns
0 on success, or -1 on formatting errors or overflow boundaries.

Clean Up the Host Bits and Prefix

int `ct_clean_cidrs`(CtCidrs *cidrs)

Sanitizes a all the cidrs in in the list. Clamps illegal prefixes and zeroes out any host bits.

Note: - Unrecognized families are ignored. - Empty lists are not an error, but cidrs being a nullptr is.

Parameters

- `cidrs` – The array of cidrs to be "cleaned"

Returns
0 on success, or -1 on invalid parameters.

int `ct_clean_cidr`(CtCidr *cidr)

Sanitizes one cidr. Clamps illegal prefixes and zeroes out any host bits.

Parameters

- `cidrs` – The array of cidrs to be "cleaned"

Returns
0 on success, or -1 on invalid parameters.

Compact A List of CIDRs to the Minimal Number

int `ct_compact`(CtCidrs *cidrs)

Compact a list of cidr blocks to the smallest number of cidr blocks.

Does in place compacting of the CtCidrs. If cidr blocks can be merged into larger blocks (smaller prefixes) then the number of blocks is reduced. If the blocks are able to be compacted, then cidrs->count will be reduced and the memory cidrs->blocks adjusted acordingly.

All the cidr blocks must be the same IP family - either IPv4 or IPv6

Parameters

- `cidrs` – The list of cidr_blocks to be compacted

Returns
-1 on error, otherwise 0.

DNS Convenience

int `ct_ip_str_to_hostname`(const char *ip, char *hostname)

Convenience function to do a direct reverse dns (PTR) lookup of an IP string.

Parameters

- `ip` – The ip address to lookup.
- `hostname` – The fully qualified hostname (FQDN) or empty string if none found. hostname must be at least NS_MAXDNAME characters long.

Returns
0 on success, -1 if any issues.

int `ct_hostname_to_address`(const char *hostname, CtCidrs *cidrs)

Performs a forward DNS lookup for a hostname and populates a CtCidrs list. Each returned address block will have its network prefix explicitly set to 32 (for IPv4) or 128 (for IPv6).

The caller is responsible for passing an initialized, clean CtCidrs structure where blocks can be dynamically allocated.

Parameters

- `hostname` – The string domain/hostname to look up.
- `cidrs` – Pointer to the destination CtCidrs structure.

Returns

0 on success, or -1 on failure/invalid resolutions.

Exclude one List of CIDRs from another list of CIDRs

int `ct_exclude_cidrs`(CtCidrs *all, CtCidrs *excluded)

Remove a set of cidr blocks from an array of cidrs.

Parameters

- `all` – The list of cidrs to be modified in place. The modifed array will have all cidrs in the *excluded* list removed.
- `excludewd` – The list of cidrs to be exluded from the full list.

Returns

0 on success, -1 otherwise.

IP Family Checks

bool `ct_is_ipv4`(const CtCidr *cidr)

Check if a cidr belongs to the IPv4 address family.

Parameters

- `cidr` – The cidr to validate.

Returns

true if the cidr is IPv4

bool `ct_is_ipv6`(const CtCidr *cidr)

Check if an cidr belongs to the IPv6 address family.

Parameters

- `cidr` – The cidr to validate.

Returns

true if the cidr is IPv6

Determine The IP That Is Num IPs After Another IP

int `ct_ip_address_increment`(const CtAddress *addr, size_t num, CtAddress *addr_inc)

Find an IP address that is an incremental number beyond an IP address.

Parameters

- `addr` – The starting address.
- `num` – The number of IP address to add to the starting address.
- `addr_inc` – The resultant IP address *num* IPs after the provided first IP address.

Returns

0 on success, or -1 on error/overflow boundaries.

Convert CIDR / IP To / From Strings

int `ct_ip_address_to_str_r`(const CtAddress *ip_addr, char *buf, size_t buflen)

Format an IP address into a caller-supplied buffer. Eliminates heap allocation bottlenecks inside high-frequency execution tracks.

See also ct_ip_address_to_str() which allocates the buffer

Parameters

- `ip_addr` – The IP address to parse.
- `buf` – Destination buffer (must be at least INET6_ADDRSTRLEN bytes long).
- `buflen` – Size of provided buffer.

Returns

0 on success, or -1 on failure.

int `ct_cidr_to_str_r`(const CtCidr *cidr, char *buf, size_t buflen)

Format a CIDR block into a caller-supplied buffer.

See also ct_cidr_to_str() which allocates a buffer.

Parameters

- `cidr` – The CIDR to generate the string from.
- `buf` – Buffer for the result
- `buflen` – Buffer size. For `IPv6`, minimum is 49 bytes for `IPv4` 19 bytes

Returns

0 on success, or -1 on failure.

char *`ct_ip_address_to_str`(const CtAddress *ip_addr)

Return a formated string of the provided IP address.

The string is malloc()'ed and the caller is responsible for calling free() when non-nullptr.

See also ct_ip_address_to_str_r() where caller provides the buffer for ouput.

- Example of a returned string: "10.0.0.22"

`IP` can be `IPv4` or `IPv6`.

Note: caller is responsible for free()ing the returned pointer.

Parameters

- `ip_addr` – The ip address to get in string form.

Returns
A malloc'ed string of the ip address. nullptr if non-IP or empty imput.

char *ct_cidr_to_str(const CtCidr *cidr)

Returns a formatted string of the CIDR block.

Result is malloc()'ed and caller must free(). See also ct_cidr_to_str_r() which uses caller provided buffer.

Cidr can be IPv4 or IPv6.

Example of a returned string: "10.0.0.9./24"

Note: The caller is responsible for using free() on the returned pointer.

Parameters

- cidr – The ip address to get in string form.

Returns
A malloc'ed string of the ip address. nullptr if non-IP or empty imput.

int ct_str_to_cidr_block(const char *str, CtCidr *cidr)

Parses a text string into a CtCidr. Can be IPv4 or IPv6.

For example "192.168.1.50/24".

Parameters

- str – The string to be parsed.
- cidr – The resultant CtCidr.

Returns
0 on success, or -1 if the string is invalid or not a valid cidr.

int ct_str_to_ip_address(const char *address, CtAddress *ip_addr)

Parses an IP string into a CtAddress structure. String can be any valid numeric IPv4 or IPv6 representation.

Parameters

- address – The IP address string to parse.
- ip_addr – The resultant CtAddress structure.

Returns
0 on success, or -1 if the string is an invalid IP address.

Get the Number of IPs In a CIDR block

size_t ct_num_ips(const CtCidr *cidr)

Returns the number of IPs in a CIDR block.

Note that IPv6 will overflow 64 bit integer for any prefix 64 or lower.

Parameters

- cidr – The cidr block to examine.

Returns
The number of IPs in the block. Primarily useful for IPv4 Since IPv6 ranges are large, SIZE_MAX will be returned if the result overflows a 64-bit unsigned integer.

CIDR To/From IP Ranges

int `ct_range_to_cidrs`(const CtAddress *first, const CtAddress *last, CtCidrs *cidrs)

Create a list of cidr blocks that fully span an IP range.

Parameters

- `first` – The first IP in the range
- `last` – The last IP in the range
- `cidrs` – The list of cidrs blocks that span the requested IP range.

Returns

0 if success, -1 otherwise.

int `ct_cidr_to_range`(const CtCidr *cidr, CtAddress *first, CtAddress *last)

Calculates the first and last IP addresses of a cidr block subnet.

Parameters

- `cidr` – The cidr block to examine.
- `first` – The first IP address in the cidr.
- `last` – The last IP address in the cidr.

Returns

0 on success, or -1 on invalid input.

int `ct_cidr_to_range_mid`(const CtCidr *cidr, CtAddress *first, CtAddress *mid, CtAddress *last)

Calculates the first, middle and last IP addresses of a cidr block subnet.

Parameters

- `cidr` – The cidr block to examine.
- `first` – The first IP address in the cidr.
- `mid` – The middle IP address in the cidr.
- `last` – The last IP address in the cidr.

Returns

0 on success, or -1 on invalid input.

int `ct_ip_address_range`(const CtAddress *addr, uint8_t prefix, CtAddress *first, CtAddress *last)

Computes the first and last IP addresses of a cidr block given an IP address and a prefix.

Same functionality as ct_cidr_to_range() which this uses.

Parameters

- `addr` – The IP address to consider
- `prefix` – The prefix of the subnet.
- `first` – The first IP address in the range.
- `last` – The last IP address in the range.

Returns

0 on success, or -1 on invalid input.

Set CIDR Prefix

int `ct_cidr_set_prefix`(CtCidr *cidr, uint8_t prefix)

Update the prefix of a cidr block. This updates in place and zeroes out any host bits past the prefix length.

The new prefix must satisfy the IP family limits. An IPv4 cannot be changed into an IPv6 by setting a prefix longer than 32 which will lead to an error.

Parameters

- `cidr` – The cidr whose prefix is to be modified.
- `prefix` – The new prefix

Returns

0 on success, or -1 otherwise. Errors can be from bad invalid input or the prefix exceeds protocol limits.

Sort A List Of CIDRs

int `ct_sort`(CtCidrs *cidrs)

In-place sort of a list of cidrs.

Parameters

- `cidrs` – The cidr blocks to sort. Since this is done in place, the blocks are moved around within the CtCidrs structure.

Returns

0 on success, -1 otherwise.

Split Cidrs into separate IPv4 and IPv6

int `ct_split_by_family`(CtCidrs *cidrs, CtCidrs *cidrs_v4, CtCidrs *cidrs_v6)

Takes a list of cidrs and returns lists of `IPv4` and `IPv6` cidrs.

Caller responsible for freeing the memory using ct_free_cidrs().

Parameters

- `cidrs` – The input list of cidrs
- `cidrs_v4` – The `IPv4` cidrs
- `cidrs_v6` – The `IPv6` cidrs

Returns

0 on success, otherwise -1

Split CIDR Into Smaller Subnets

static void `add_v6_bit_step_32`(uint32_t *addr32, uint8_t prefix_len)

Increments an IPv6 address using high-performance 32-bit native arithmetic blocks. Replaces costly byte-loop array adjustments with direct CPU register bit strides.

CtCidrs *`ct_subnets_split`(const CtCidr *cidr, uint8_t prefix)

Splits a cidr into a list of smaller subnets of with prefix lengths 'prefix'. The provided prefix must be larger then cidr->prefix (obviously).

Caller is responsible for freeing the memory of the resulting list of cidrs. Both the result->blocks as well as the CtCidrs structure itself.

Parameters

- `cidr` – The cidr to split into smaller subnets.
- `prefix` – The prefix to use.

Returns

A pointer to a new heap-allocated CtCidrs holding the array of cidrs. Or a nullptr on failure.

Memory Management

void `ct_free_cidrs`(CtCidrs *cidrs)

Frees memory allocations inside a CtCidrs struct.

Resets internal pointer to nullptr and count to 0.

Parameters

- `cidrs` – Pointer to the CtCidrs

Returns

nothing - it's a void function

bool `ct_add_cidr_to_cidrs`(CtCidrs *cidrs, const CtCidr *cidr)

Add a cidr block to the list of cidrs.

Parameters

- `cidrs` – Add the new cidr to this list of cidrs.
- `cidr` – The cidr to add

Returns

true on success, false on error

bool `ct_allocate_cidrs`(size_t count, CtCidrs *cidrs)

Allocate 'count' cidr blocks cidrs.

Note new memory is set to zero

Parameters

- `count` – Allocate `count` blocks
- `cidrs` – The cidr list to modify.

Returns

true on success, false on error

Get Cidrtools Version

char *ct_version(void)

Returns the current version

Returns

Returns the current version string.

INDEX